

PREMIER UNIVERSITY, CHITTAGONG
Department of Computer Science and Engineering (CSE)



MOVIELENS DATA ANALYSIS USING HADOOP

A Distributed Data Processing Project Using MapReduce and Apache Pig

COURSE NAME: Big Data Analytics Laboratory	
COURSE CODE: CSE 4346	
SEMESTER: 8th	BATCH: 42nd
SESSION: Spring 2026	SECTION: B
DATE OF SUBMISSION: 30-08-2026	

NAME	Tahmid Jamal
ID NUMBER	0222220005101083

Submitted To:	MD Tamim Hossain Lecturer, Department of Computer Science and Engineering
----------------------	---

Abstract

This project demonstrates how distributed data-processing technologies can be used to analyze a large movie-rating dataset. The system uses Hadoop MapReduce Java programs for focused batch operations and Apache Pig Latin scripts for relational analysis, joins, grouping, aggregation, sorting, and segmentation. The input consists of 3,883 movies, 1,000,209 ratings, and 6,040 users. The workflows calculate movie-level average ratings, identify highly rated records, count genre representation, join ratings with user demographics, analyze rating trends by year, rank popular movies, compare demographic groups, and classify users by review activity. The project illustrates the value of separating raw data ingestion, transformation, aggregation, and persistence into repeatable distributed workflows.

Table of Contents

- 1. Introduction 3
- 2. Problem Statement and Objectives 3
- 3. Dataset Description 3
- 4. System Architecture and Methodology 4
- 5. MapReduce Implementation 4
 - 5.1 Average Rating 4
 - 5.2 High Rating Filter 7
 - 5.3 Genre Counter 9
 - 5.4 User Rating Join 11
- 6. Apache Pig Implementation 13
 - 6.1 Top_Movies 14
 - 6.2 Genre_breakdown 15
 - 6.3 Demographics_analysis 16
 - 6.4 Rating_trends 17
 - 6.5 User_behavior 18
- 7. Conclusion 19

1. Introduction

Modern movie platforms generate large volumes of user, title, and rating data. Processing these records on a single machine can become slow, difficult to scale, and less resilient as data volume grows. Distributed frameworks address this challenge by splitting computation into parallel tasks and combining the intermediate results.

This project applies Hadoop MapReduce and Apache Pig to a MovieLens dataset. MapReduce provides explicit control over mapper and reducer behavior for targeted operations. Pig provides a higher-level data-flow language for multi-step transformations and analytical queries. Together, the two approaches show how the same data can support both low-level distributed programming and concise data-flow analysis.

2. Problem Statement and Objectives

The central problem is to extract useful movie and user insights from CSV files containing millions of rating events while preserving a workflow that can run in a Hadoop environment. The project objectives are:

- 1. Compute the average rating and rating count for each movie identifier.
- 2. Filter rating events with a score of at least 4.0.
- 3. Count how often each genre appears in the movie catalog.
- 4. Join ratings with user gender, age, occupation, and ZIP-code data.
- 5. Identify highly rated movies while requiring more than five ratings.
- 6. Measure rating activity and average ratings by calendar year.
- 7. Compare rating behavior across gender and age groups.
- 8. Segment users into casual, moderate, and heavy reviewers.

3. Dataset Description

The project uses three comma-separated input files. Each script removes the header row before casting fields to numeric types.

Dataset File	Main Fields	Records
movies.csv	movieId, title, genres	3,883 movies
ratings.csv	userId, movieId, rating, timestamp	1,000,209 ratings
users.csv	userId, gender, age, occupation, zip	6,040 users

The rating values are on the familiar five-point scale. Timestamps are Unix timestamps and are converted to years in the rating-trend analysis. The supplied data covers rating events from 2000 through 2003. User records contain coded demographic values; the scripts retain those codes rather than attempting to interpret them as descriptive labels.

4. System Architecture and Methodology

The workflow follows a standard distributed analytics pipeline:

1. Store the CSV files in the Hadoop input directory at /movielens/input.
2. Load raw records with Hadoop-compatible input formats.
3. Remove headers and null identifiers before conversion.
4. Cast identifiers, ratings, ages, and timestamps to analytical types.
5. Apply map, filter, join, group, aggregate, order, and limit operations.
6. Persist results to HDFS output directories using PigStorage.

The Pig scripts explicitly configure HDFS at `hdfs://localhost:9000` and write outputs under /movielens. This makes the intended deployment environment clear and separates source input data from derived analytical results.

5. MapReduce Implementation

Four Java programs implement focused MapReduce jobs. Each program skips CSV headers and uses Hadoop Writable types for keys and values.

Program	Processing Logic
AverageRating.java	Maps movie IDs to ratings and reduces each group to an average and total count.
GenreCounter.java	Splits pipe-delimited genre strings and counts each genre; a combiner reduces intermediate traffic.
HighRatingFilter.java	Performs map-only filtering and emits records where rating is at least 4.0.
UserRatingJoin.java	Loads users.csv through the distributed cache and enriches each rating with demographic information.

5.1 Average Rating

Maps movie IDs to ratings and reduces each group to an average and total count.

Driver class:

```
AverageRating.java
1  import java.io.IOException;
2  import org.apache.hadoop.conf.Configuration;
3  import org.apache.hadoop.fs.Path;
4  import org.apache.hadoop.io.FloatWritable;
5  import org.apache.hadoop.io.LongWritable;
6  import org.apache.hadoop.io.Text;
7  import org.apache.hadoop.mapreduce.Job;
8  import org.apache.hadoop.mapreduce.Mapper;
9  import org.apache.hadoop.mapreduce.Reducer;
10 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
11 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
12
13 public class AverageRating {
14
15     public static class RatingMapper extends Mapper<LongWritable, Text, Text, FloatWritable> { 1 usage
16         private Text movieId = new Text(); 2 usages
17         private FloatWritable rating = new FloatWritable(); 2 usages
18
19         @Override no usages
20         public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
21             String line = value.toString();
22             if (line.startsWith("user:")) return; // Skip CSV header
23             String[] tokens = line.split(",");
24             if (tokens.length >= 3) {
25                 movieId.set(tokens[1]);
26                 rating.set(Float.parseFloat(tokens[2]));
27                 context.write(movieId, rating);
28             }
29         }
30     }
31 }
```

```

public static class RatingReducer extends Reducer<Text, FloatWritable, Text, Text> { 1
    @Override no usages
    public void reduce(Text key, Iterable<FloatWritable> values, Context context)
        throws IOException, InterruptedException {
        float sum = 0;
        int count = 0;
        for (FloatWritable val : values) {
            sum += val.get();
            count++;
        }
        float avg = sum / count;
        context.write(key, new Text("AvgRating: " + avg + "\tTotalCount: " + count));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Rating Per Movie");
    job.setJar("MapReduceJobs.jar");
    job.setMapperClass(RatingMapper.class);
    job.setReducerClass(RatingReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(FloatWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

Mapper Class:

```

5
6 import java.io.IOException;
7 import org.apache.hadoop.io.FloatWritable;
8 import org.apache.hadoop.io.LongWritable;
9 import org.apache.hadoop.io.Text;
10 import org.apache.hadoop.mapreduce.Mapper;
11
12 public class AverageRating$RatingMapper extends Mapper<LongWritable, Text, Text, FloatWritable> {
13     private Text movieId = new Text();
14     private FloatWritable rating = new FloatWritable();
15
16     public AverageRating$RatingMapper() {
17     }
18
19     public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, Text, FloatWritable>.Context var3)
20         throws IOException, InterruptedException {
21         String var4 = var2.toString();
22         if (!var4.startsWith("userId")) {
23             String[] var5 = var4.split(",");
24             if (var5.length >= 3) {
25                 this.movieId.set(var5[1]);
26                 this.rating.set(Float.parseFloat(var5[2]));
27                 var3.write(this.movieId, this.rating);
28             }
29         }
30     }
31 }

```

Reducer Class:

```
AverageRating.java  AverageRating$RatingMapper.class  AverageRating$RatingReducer.class x
Decompiled .class file, bytecode version: 55.0 (Java 11)

4  //
5
6  import java.io.IOException;
7  import org.apache.hadoop.io.FloatWritable;
8  import org.apache.hadoop.io.Text;
9  import org.apache.hadoop.mapreduce.Reducer;
10
11 public class AverageRating$RatingReducer extends Reducer<Text, FloatWritable, Text, Text> {
12     public AverageRating$RatingReducer() {
13     }
14
15     public void reduce(Text var1, Iterable<FloatWritable> var2, Reducer<Text, FloatWritable,
16         float var4 = 0.0F;
17         int var5 = 0;
18
19         for(FloatWritable var7 : var2) {
20             var4 += var7.get();
21             ++var5;
22         }
23
24         float var8 = var4 / (float)var5;
25         var3.write(var1, new Text("AvgRating: " + var8 + "\tTotalCount: " + var5));
26     }
27 }
28
```

Execution:

```
C:\Users\jamal\Downloads\archive\mapreduce\hadoop jar MapReduceJobs.jar AverageRating /movielens/input/ratings.csv /movielens/mr_out1
2026-09-08 00:46:35,833 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-08 00:46:36,312 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-08 00:46:36,339 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/jamal/.staging/job_1788805427938_0002
2026-09-08 00:46:36,664 INFO input.FileInputFormat: Total input files to process : 1
2026-09-08 00:46:36,765 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-08 00:46:36,999 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1788805427938_0002
2026-09-08 00:46:37,002 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-08 00:46:37,282 INFO conf.Configuration: resource-types.xml not found
2026-09-08 00:46:37,282 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-08 00:46:37,483 INFO impl.YarnClientImpl: Submitted application application_1788805427938_0002
2026-09-08 00:46:37,554 INFO mapreduce.Job: The url to track the job: http://LAPTOP-GCKBVH8:8088/proxy/application_1788805427938_0002/
2026-09-08 00:46:37,554 INFO mapreduce.Job: Running job: job_1788805427938_0002
2026-09-08 00:46:47,757 INFO mapreduce.Job: Job job_1788805427938_0002 running in uber mode : false
2026-09-08 00:46:47,757 INFO mapreduce.Job:  map 0% reduce 0%
2026-09-08 00:46:55,898 INFO mapreduce.Job:  map 100% reduce 0%
2026-09-08 00:47:03,010 INFO mapreduce.Job:  map 100% reduce 100%
2026-09-08 00:47:03,013 INFO mapreduce.Job: Job job_1788805427938_0002 completed successfully
2026-09-08 00:47:03,111 INFO mapreduce.Job: Counters: 54
```

Output:

1	AvgRating: 4.1468463	TotalCount: 2077
10	AvgRating: 3.5405405	TotalCount: 888
100	AvgRating: 3.0625	TotalCount: 128
1000	AvgRating: 3.05	TotalCount: 20
1002	AvgRating: 4.25	TotalCount: 8
1003	AvgRating: 2.9421487	TotalCount: 121
1004	AvgRating: 2.6633663	TotalCount: 101
1005	AvgRating: 2.3732395	TotalCount: 142

5.2 High Rating Filter

Performs map-only filtering and emits records where rating is at least 4.0

Driver Class:

```
HighRatingFilter.java x
1  import java.io.IOException;
2  import org.apache.hadoop.conf.Configuration;
3  import org.apache.hadoop.fs.Path;
4  import org.apache.hadoop.io.LongWritable;
5  import org.apache.hadoop.io.NullWritable;
6  import org.apache.hadoop.io.Text;
7  import org.apache.hadoop.mapreduce.Job;
8  import org.apache.hadoop.mapreduce.Mapper;
9  import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
10 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
11
12 public class HighRatingFilter {
13
14     public static class FilterMapper extends Mapper<LongWritable, Text, NullWritable, Text> {
15         @Override
16         public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
17             String line = value.toString();
18             if (line.startsWith("userId")) return;
19             String[] tokens = line.split(",");
20             if (tokens.length >= 3) {
21                 float rating = Float.parseFloat(tokens[2]);
22                 if (rating >= 4.0) {
23                     context.write(NullWritable.get(), value);
24                 }
25             }
26         }
27     }
28
29     public static void main(String[] args) throws Exception {
30         Configuration conf = new Configuration();
31         Job job = Job.getInstance(conf, "Filter High Ratings");
32         job.setJar("MapReduceJobs.jar");
33         job.setMapperClass(FilterMapper.class);
34         job.setNumReduceTasks(0); // Map-Only Execution
35         job.setOutputKeyClass(NullWritable.class);
36         job.setOutputValueClass(Text.class);
37         FileInputFormat.addInputPath(job, new Path(args[0]));
38         FileOutputFormat.setOutputPath(job, new Path(args[1]));
39         System.exit(job.waitForCompletion(true) ? 0 : 1);
40     }
41 }
```

Mapper class:

```
import java.io.IOException;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class HighRatingFilter$FilterMapper extends Mapper<LongWritable, Text, NullWritable, Text> {
    public HighRatingFilter$FilterMapper() {
    }

    public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, NullWritable, Text>.Context var3) throws IOException, InterruptedException {
        String var4 = var2.toString();
        if (!var4.startsWith("userId")) {
            String[] var5 = var4.split(",");
            if (var5.length >= 3) {
                float var6 = Float.parseFloat(var5[2]);
                if ((double)var6 >= (double)4.0F) {
                    var3.write(NullWritable.get(), var2);
                }
            }
        }
    }
}
```

Reducer Class:

```
import java.io.IOException;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class HighRatingFilter$FilterMapper extends Mapper<LongWritable, Text, NullWritable, Text> {
    public HighRatingFilter$FilterMapper() {
    }

    public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, NullWritable, Text>.Context var3) throws IOException, InterruptedException {
        String var4 = var2.toString();
        if (!var4.startsWith("userId")) {
            String[] var5 = var4.split(",");
            if (var5.length >= 3) {
                float var6 = Float.parseFloat(var5[2]);
                if (((double)var6 >= (double)4.0F) {
                    var3.write(NullWritable.get(), var2);
                }
            }
        }
    }
}
```

Execution:

```
C:\Users\jama\Downloads\archive\mapreduce>hadoop jar MapReduceJobs.jar HighRatingFilter /movielens/input/ratings.csv /movielens/mr_out2
2026-09-08 00:48:00,655 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-08 00:48:01,015 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-08 00:48:01,029 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/jama/.staging/job_1788805427938_0003
2026-09-08 00:48:01,261 INFO input.FileInputFormat: Total input files to process : 1
2026-09-08 00:48:01,339 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-08 00:48:01,515 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1788805427938_0003
2026-09-08 00:48:01,520 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-08 00:48:01,744 INFO conf.Configuration: resource-types.xml not found
2026-09-08 00:48:01,744 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-08 00:48:01,916 INFO impl.YarnClientImpl: Submitted application application_1788805427938_0003
2026-09-08 00:48:01,978 INFO mapreduce.Job: The url to track the job: http://LAPTOP-GCKDVH8:8088/proxy/application_1788805427938_0003/
2026-09-08 00:48:01,978 INFO mapreduce.Job: Running job: job_1788805427938_0003
2026-09-08 00:48:11,070 INFO mapreduce.Job: Job job_1788805427938_0003 running in uber mode : false
2026-09-08 00:48:11,070 INFO mapreduce.Job: map 0% reduce 0%
2026-09-08 00:48:17,180 INFO mapreduce.Job: map 100% reduce 0%
2026-09-08 00:48:18,198 INFO mapreduce.Job: Job job_1788805427938_0003 completed successfully
2026-09-08 00:48:18,297 INFO mapreduce.Job: Counters: 33
```

Output:

File contents

```
1,1193,5,978300760
1,3408,4,978300275
1,2355,5,978824291
1,1287,5,978302039
1,2804,5,978300719
1,594,4,978302268
1,919,4,978301368
1,595,5,978824268
```

5.3 Genre Counter

Splits pipe-delimited genre strings and counts each genre; a combiner reduces intermediate traffic

Driver class:

```
GenreCounter.java x GenreCounter.class

1  > import ...
12
13  public class GenreCounter {
14
15      public static class GenreMapper extends Mapper<LongWritable, Text, Text, IntWritable> { 1 usage
16          private final static IntWritable one = new IntWritable(1); 1 usage
17          private Text genreText = new Text(); 2 usages
18
19          @Override no usages
20          public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
21              String line = value.toString();
22              if (line.startsWith("movieId")) return;
23              String[] tokens = line.split(regex: "\\|");
24              if (tokens.length >= 3) {
25                  String genresStr = tokens[tokens.length - 1]; // Handle titles containing commas
26                  String[] genres = genresStr.split(regex: "\\|");
27                  for (String g : genres) {
28                      if (g.trim().isEmpty()) {
29                          genreText.set(g.trim());
30                          context.write(genreText, one);
31                      }
32                  }
33              }
34          }
35      }
36
37      public static class GenreReducer extends Reducer<Text, IntWritable, Text, IntWritable> { 2 usages
38          @Override no usages
39          public void reduce(Text key, Iterable<IntWritable> values, Context context)
40              throws IOException, InterruptedException {
41              int sum = 0;
42              for (IntWritable val : values) {
43                  sum += val.get();
44              }
45              context.write(key, new IntWritable(sum));
46          }
47      }
48
49      public static void main(String[] args) throws Exception {
50          Configuration conf = new Configuration();
51          Job job = Job.getInstance(conf, "Genre Counter");
52          job.setJar("MapReduceJobs.jar");
53          job.setMapperClass(GenreMapper.class);
54          job.setCombinerClass(GenreReducer.class);
55          job.setReducerClass(GenreReducer.class);
56          job.setOutputKeyClass(Text.class);
```

Mapper Class:

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class GenreCounter$GenreMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    private static final IntWritable one = new IntWritable(1);
    private Text genreText = new Text();

    public GenreCounter$GenreMapper() {
    }

    public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, Text, IntWritable>.Context var3) throws IOException, InterruptedException {
        String var4 = var2.toString();
        if (!var4.startsWith("movieId")) {
            String[] var5 = var4.split(",");
            if (var5.length >= 3) {
                String var6 = var5[var5.length - 1];
                String[] var7 = var6.split("\\|");

                for (String var11 : var7) {
                    if (!var11.trim().isEmpty()) {
                        this.genreText.set(var11.trim());
                        var3.write(this.genreText, one);
                    }
                }
            }
        }
    }
}
```

Reducer Class:

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class GenreCounter$GenreReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public GenreCounter$GenreReducer() {
    }

    public void reduce(Text var1, Iterable<IntWritable> var2, Reducer<Text, IntWritable, Text, IntWritable>.Context var3) throws IOException, InterruptedException {
        int var4 = 0;

        for(IntWritable var6 : var2) {
            var4 += var6.get();
        }

        var3.write(var1, new IntWritable(var4));
    }
}
```

Execution:

```
... (Users\jamal\Downloads\archive\mapreduce>hadoop jar MapReduceJobs.jar GenreCounter /movieiens/input/movies.csv /movieiens/GenreCounter_out
2026-09-08 00:58:39,450 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-08 00:58:39,817 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-09-08 00:58:39,834 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/jamal/.staging/job_1788805427938_0007
2026-09-08 00:58:40,067 INFO input.FileInputFormat: Total input files to process : 1
2026-09-08 00:58:40,150 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-08 00:58:40,324 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1788805427938_0007
2026-09-08 00:58:40,328 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-08 00:58:40,550 INFO conf.Configuration: resource-types.xml not found
2026-09-08 00:58:40,550 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-08 00:58:40,643 INFO impl.YarnClientImpl: Submitted application application_1788805427938_0007
2026-09-08 00:58:40,700 INFO mapreduce.job: The url to track the job: http://LAPTOP-GCKBVH#8:8088/proxy/application_1788805427938_0007/
2026-09-08 00:58:40,700 INFO mapreduce.job: Running job: job_1788805427938_0007
2026-09-08 00:58:49,843 INFO mapreduce.job: Job job_1788805427938_0007 running in uber mode : false
2026-09-08 00:58:49,843 INFO mapreduce.job: map 0% reduce 0%
2026-09-08 00:58:55,952 INFO mapreduce.job: map 100% reduce 0%
2026-09-08 00:59:02,047 INFO mapreduce.job: map 100% reduce 100%
2026-09-08 00:59:02,049 INFO mapreduce.job: Job job_1788805427938_0007 completed successfully
2026-09-08 00:59:02,148 INFO mapreduce.job: Counters: 54
```

Output:

Action	503
Adventure	283
Animation	105
Children's	251
Comedy	1200
Crime	211
Documentary	127
Drama	1603

5.4 User Rating Join

Loads users.csv through the distributed cache and enriches each rating with demographic information

Driver Class:

```
> import ...

public class UserRatingJoin {

    public static class JoinMapper extends Mapper<LongWritable, Text, Text, Text> { 1 usage
        private Map<String, String> userMap = new HashMap<>(); 2 usages

        @Override no usages
        protected void setup(Context context) throws IOException, InterruptedException {
            URI[] cacheFiles = context.getCacheFiles();
            if (cacheFiles != null && cacheFiles.length > 0) {
                // FIXED LINE: Extract only the simple file name (e.g. "users.csv")
                String fileName = new Path(cacheFiles[0].getPath()).getName();
                File cacheFile = new File(fileName);

                try (BufferedReader reader = new BufferedReader(new FileReader(cacheFile))) {
                    String line;
                    while ((line = reader.readLine()) != null) {
                        if (line.startsWith("userId")) continue;
                        String[] tokens = line.split(regex: " ");
                        if (tokens.length >= 3) {
                            userMap.put(tokens[0], tokens[1] + "," + tokens[2]);
                        }
                    }
                }
            }
        }

        @Override no usages
        public void map(LongWritable key, Text value, Context context) throws IOException, InterruptedException {
            String line = value.toString();
            if (line.startsWith("userId")) return;
            String[] tokens = line.split(regex: " ");
            if (tokens.length >= 3) {
                String userId = tokens[0];
                String movieId = tokens[1];
                String rating = tokens[2];

                String userInfo = userMap.getOrDefault(userId, defaultValue: "Unknown,Unknown");
                context.write(new Text(userId), new Text("MovieID:" + movieId + " | Rating:" + rating + " | Demographics(Gender,Age):" + userInfo));
            }
        }
    }
}
```

Mapper Class:

```
0      import java.net.URI;
1      import java.util.HashMap;
2      import java.util.Map;
3      import org.apache.hadoop.fs.Path;
4      import org.apache.hadoop.io.LongWritable;
5      import org.apache.hadoop.io.Text;
6      import org.apache.hadoop.mapreduce.Mapper;
7
8      public class UserRatingJoin$JoinMapper extends Mapper<LongWritable, Text, Text, Text> {
9          private Map<String, String> userMap = new HashMap();
10
11          public UserRatingJoin$JoinMapper() {
12          }
13
14          protected void setup(Mapper<LongWritable, Text, Text, Text>.Context var1) throws IOException, InterruptedException {
15              URI[] var2 = var1.getCacheFiles();
16              if (var2 != null && var2.length > 0) {
17                  String var3 = (new Path(var2[0].getPath())).getName();
18                  File var4 = new File(var3);
19
20                  String var6;
21                  try (BufferedReader var5 = new BufferedReader(new FileReader(var4))) {
22                      while((var6 = var5.readLine()) != null) {
23                          if (!var6.startsWith("userId")) {
24                              String[] var7 = var6.split(",");
25                              if (var7.length >= 3) {
26                                  this.userMap.put(var7[0], var7[1] + "," + var7[2]);
27                              }
28                          }
29                      }
30                  }
31              }
32          }
33
34          public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, Text, Text>.Context var3) throws IOException, InterruptedException {
35              String var4 = var2.toString();
36              if (!var4.startsWith("userId")) {
37                  String[] var5 = var4.split(",");
38                  if (var5.length >= 3) {
39                      String var6 = var5[0];
40                      String var7 = var5[1];
41                      String var8 = var5[2];
42                      String var9 = (String)this.userMap.getOrDefault(var6, "Unknown,Unknown");
```

Reducer Class:

```
import java.io.IOException;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class HighRatingFilter$FilterMapper extends Mapper<LongWritable, Text, NullWritable, Text> {
    public HighRatingFilter$FilterMapper() {
    }

    public void map(LongWritable var1, Text var2, Mapper<LongWritable, Text, NullWritable, Text>.Context var3) throws IOException, InterruptedException {
        String var4 = var2.toString();
        if (!var4.startsWith("userId")) {
            String[] var5 = var4.split(",");
            if (var5.length >= 3) {
                float var6 = Float.parseFloat(var5[2]);
                if ((double)var6 >= (double)4.0F) {
                    var3.write(NullWritable.get(), var2);
                }
            }
        }
    }
}
```

Execution:

```
C:\Users\jamal\Downloads\archive\mapreduce>hadoop jar mapreduce\jobs.jar userRatingJoin /movielens/input/users.csv /movielens/input/ratings.csv /movielens/userRatingJoin_out
2026-09-08 00:57:12,757 INFO client.RMPProxy: Connecting to ResourceManager at /0.0.0.0:8032
2026-09-08 00:57:13,207 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with
2026-09-08 00:57:13,232 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/jamal/.staging/job_1788805427938_0006
2026-09-08 00:57:13,556 INFO input.FileInputFormat: Total input files to process : 1
2026-09-08 00:57:13,673 INFO mapreduce.JobSubmitter: number of splits:1
2026-09-08 00:57:13,912 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1788805427938_0006
2026-09-08 00:57:13,912 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-09-08 00:57:14,190 INFO conf.Configuration: resource-types.xml not found
2026-09-08 00:57:14,190 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-09-08 00:57:14,273 INFO impl.YarnClientImpl: Submitted application application_1788805427938_0006
2026-09-08 00:57:14,340 INFO mapreduce.Job: The url to track the job: http://LAPTOP-GCKBVH8:8088/proxy/application_1788805427938_0006/
2026-09-08 00:57:14,340 INFO mapreduce.Job: Running job: job_1788805427938_0006
2026-09-08 00:57:23,516 INFO mapreduce.Job: Job job_1788805427938_0006 running in uber mode : false
2026-09-08 00:57:23,516 INFO mapreduce.Job: map 0% reduce 0%
2026-09-08 00:57:30,642 INFO mapreduce.Job: map 100% reduce 0%
2026-09-08 00:57:30,644 INFO mapreduce.Job: Job job_1788805427938_0006 completed successfully
```

Output:

```
1  MovieID:1193 | Rating:5 | Demographics(Gender,Age):F,1
1  MovieID:661 | Rating:3 | Demographics(Gender,Age):F,1
1  MovieID:914 | Rating:3 | Demographics(Gender,Age):F,1
1  MovieID:3408 | Rating:4 | Demographics(Gender,Age):F,1
1  MovieID:2355 | Rating:5 | Demographics(Gender,Age):F,1
1  MovieID:1197 | Rating:3 | Demographics(Gender,Age):F,1
1  MovieID:1287 | Rating:5 | Demographics(Gender,Age):F,1
1  MovieID:2804 | Rating:5 | Demographics(Gender,Age):F,1
```

6. Apache Pig Implementation

Five Pig Latin scripts provide higher-level analytical workflows.

Script	Purpose and Output	Output Location
top_movies.pig	Calculates movie averages, keeps movies with more than five ratings, joins titles, and stores the top ten by average rating.	/movielens/pig_out1
genre_breakdown.pig	Joins movies and ratings, tokenizes genres, and stores rating volume and average rating by genre.	/movielens/pig_out3
demographics_analysis.pig	Joins users and ratings, then stores average rating and rating count by gender and by age group.	/movielens pig_out2_age&gender
rating_trends.pig	Converts Unix timestamps to years and stores annual review volume and average rating.	/movielens/pig_out_tr ends
user_behavior.pig	Calculates per-user review count and average rating, then groups users into engagement tiers.	/movielens/pig_out_us er_segments

6.1 Top_Movies

Script:

```
-- Load data as raw chararray to safely filter headers
movies_raw = LOAD 'hdfs://localhost:9000/movielens/input/movies.csv' USING PigStorage(',')
AS (movieId:chararray, title:chararray, genres:chararray);

ratings_raw = LOAD 'hdfs://localhost:9000/movielens/input/ratings.csv' USING PigStorage(',')
AS (userId:chararray, movieId:chararray, rating:chararray, timestamp:chararray);

-- Step 1: Filter out the CSV headers
movies_filtered = FILTER movies_raw BY movieId != 'movieId' AND movieId IS NOT NULL;
ratings_filtered = FILTER ratings_raw BY userId != 'userId' AND userId IS NOT NULL;

-- Step 2: Safely cast columns to numeric types
movies = FOREACH movies_filtered GENERATE
    (int)movieId AS movieId,
    title AS title,
    genres AS genres;

ratings = FOREACH ratings_filtered GENERATE
    (int)userId AS userId,
    (int)movieId AS movieId,
    (Float)rating AS rating;

-- Step 3: Group & Calculate Averages
grp_ratings = GROUP ratings BY movieId;
avg_ratings = FOREACH grp_ratings GENERATE
    group AS movieId,
    AVG(ratings.rating) AS avg_rating,
    COUNT(ratings) AS num_ratings;

-- Filter for popular movies (e.g., > 5 reviews)
popular_movies = FILTER avg_ratings BY num_ratings > 5;

-- Step 4: Join with Movies
joined_data = JOIN popular_movies BY movieId, movies BY movieId;

projected_data = FOREACH joined_data GENERATE
    movies::title AS title,
    popular_movies::avg_rating AS avg_rating,
    popular_movies::num_ratings AS total_votes;

-- Step 5: Top 10 via nested ORDER (avoids TotalOrderPartitioner sampler, which fails on this cluster)
all_movies = GROUP projected_data ALL;
top10 = FOREACH all_movies {
```

Execution:

```
C:\Users\jamal\Downloads\archive\script> pig -x mapreduce top_movies.pig
2026-09-08 03:42:13,634 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-08 03:42:13,634 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-08 03:42:13,634 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-08 03:42:14,030 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-08 03:42:14,030 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig.1788817334016.log
2026-09-08 03:42:14,806 [main] INFO org.apache.pig.impl.util.Utils - Default bootstrap file C:\Users\jamal\.pigbootstrap not found
2026-09-08 03:42:14,872 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-08 03:42:14,874 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-08 03:42:15,761 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-top_movies.pig-adba04ce-23cf-41ee-b935-20685ac47ee8
2026-09-08 03:42:15,761 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-08 03:42:15,791 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,334 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,454 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,601 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,651 [main] WARN org.apache.pig.newplan.BaseOperatorPlan - Encountered Warning IMPLICIT_CAST_TO_LONG 1 time(s).
2026-09-08 03:42:16,661 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.textoutputformat.separator is deprecated. Instead, use mapreduce.output.textoutputformat.separator
2026-09-08 03:42:16,688 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: HASH_JOIN, GROUP_BY, FILTER
2026-09-08 03:42:16,709 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,725 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-08 03:42:16,772 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, SplitFilter, StreamTypeCastInserter]}
2026-09-08 03:42:16,858 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (61.01d Gen) of size 1048576000 to monitor. collectionUsageThreshold = 734003200, usageThreshold = 734003200
2026-09-08 03:42:16,883 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:16,912 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for movies.raw: $2
2026-09-08 03:42:16,915 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for ratings.raw: $3
2026-09-08 03:42:17,011 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-09-08 03:42:17,060 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-08 03:42:17,112 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler$LastInputStreamingOptimizer - Rewrite: POPackage->POForeach to POPackage(JoinPackager)
2026-09-08 03:42:17,122 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size before optimization: 3
2026-09-08 03:42:17,122 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size after optimization: 3
2026-09-08 03:42:17,160 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:42:17,233 [main] INFO org.apache.hadoop.yarn.client.RMProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-09-08 03:42:17,549 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - yarn.resourcemanager.system-metrics-publisher.enabled is deprecated. Instead, use yarn.system-metrics-publisher.enabled
2026-09-08 03:42:17,577 [main] INFO org.apache.pig.tools.pigstats.mapreduce.MRScriptState - Pig script settings are added to the job
2026-09-08 03:42:17,592 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.reduce.markreset.buffer.percent is deprecated. Instead, use mapreduce.reduce.markreset.buffer.percent
2026-09-08 03:42:17,592 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - mapreduce.reduce.markreset.buffer.percent is not set, default is 0.3
```

Output:

```
Lamerica (1994),4.75,8
"Apple,4.666666666666667,9
Sanjuro (1962),4.608695652173913,69
Seven Samurai (The Magnificent Seven) (Shichinin no sam
(1954),4.560509554140127,628
"Shawshank Redemption,4.554557700942973,2227
"Godfather,4.524966261808367,2223
"Close Shave,4.52054794520548,657
```

6.2 Genre_breakdown

Script:

```
8  -- Load data as raw chararray to safely filter headers
9  movies_raw = LOAD 'hdfs://localhost:9000/movielens/input/movies.csv' USING PigStorage(',')
10     AS (movieId:chararray, title:chararray, genres:chararray);
11  ratings_raw = LOAD 'hdfs://localhost:9000/movielens/input/ratings.csv' USING PigStorage(',')
12     AS (userId:chararray, movieId:chararray, rating:chararray, timestamp:chararray);
13
14  movies_filtered = FILTER movies_raw BY movieId != 'movieId' AND movieId IS NOT NULL;
15  ratings_filtered = FILTER ratings_raw BY userId != 'userId' AND userId IS NOT NULL;
16
17  movies = FOREACH movies_filtered GENERATE
18     (int)movieId AS movieId,
19     title AS title,
20     genres AS genres;
21
22  ratings = FOREACH ratings_filtered GENERATE
23     (int)userId AS userId,
24     (int)movieId AS movieId,
25     (float)rating AS rating;
26
27  -- Join movies and ratings
28  movie_ratings = JOIN ratings BY movieId, movies BY movieId;
29
30  -- Flatten genres (split by '|')
31  genre_rows = FOREACH movie_ratings GENERATE
32     ratings::rating AS rating,
33     FLATTEN(TOKENIZE(movies::genres, '|')) AS genre;
34
35  -- Group by individual Genre
36  grp_genre = GROUP genre_rows BY genre;
37  genre_summary = FOREACH grp_genre GENERATE
38     group AS genre,
39     COUNT(genre_rows) AS rating_count,
40     AVG(genre_rows.rating) AS avg_rating;
41
42  -- Order by overall rating volume (nested ORDER avoids TotalOrderPartitioner)
43  all_genres = GROUP genre_summary ALL;
44  sorted_genres = FOREACH all_genres {
45     ordered = ORDER genre_summary BY rating_count DESC;
46     GENERATE FLATTEN(ordered);
47  };
48
49  -- Persist to HDFS (DUMP only prints to stdout and does not write HDFS)
50  STORE sorted_genres INTO 'hdfs://localhost:9000/movielens/pig_out3' USING PigStorage(',');
```

Execution:

```
C:\Users\jamal\Downloads\archive\script>pig -x mapreduce genre_breakdown.pig
2026-09-08 04:00:43,344 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-08 04:00:43,348 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-08 04:00:43,348 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-08 04:00:43,775 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-08 04:00:43,775 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig-1788818443759.log
2026-09-08 04:00:44,410 [main] INFO org.apache.pig.impl.util.Utils - Default bootstrap file C:\Users\jamal/.pigbootstrap not found
2026-09-08 04:00:44,488 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-08 04:00:44,488 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-08 04:00:45,106 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-genre_breakdown.pig-bbb65723-ch86-43bd-8bd2-49d3b3b65514
2026-09-08 04:00:45,106 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-08 04:00:45,122 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:00:45,677 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:00:45,848 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:00:45,946 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (61 Old Gen) of size 1048576000 to monitor. collectionUsageThreshold = 734003200, usageThreshold = 734003200
2026-09-08 04:00:46,004 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:00:46,057 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.textoutputformat.separator is deprecated. Instead, use mapreduce.output.textoutputformat.separator
2026-09-08 04:00:46,088 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: HASH_JOIN, GROUP_BY, FILTER
2026-09-08 04:00:46,120 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:00:46,120 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-08 04:00:46,167 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter]}
2026-09-08 04:00:46,215 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for ratings_raw: $3
2026-09-08 04:00:46,215 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for movies_raw: $1
2026-09-08 04:00:46,357 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-09-08 04:00:46,413 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-08 04:00:46,444 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.SecondaryKeyOptimizerMR - Using Secondary Key Optimization for MapReduce node scope-99
2026-09-08 04:00:46,450 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler$LastInputStreamingOptimizer - Rewrite: POPackage->POForEach to POPackage(JoinPackager)
```

Output:

```
Comedy,285699,3.515469077595651
Drama,261556,3.7621656547737388
Action,197290,3.436367783465964
Thriller,132021,3.516281500670348
Sci-Fi,127697,3.463237194295872
Romance,116773,3.599410822707304
Adventure,104850,3.4664377682403433
Crime,59396,3.6347902215637418
```

6.3 Demographics_analysis

Script:

```
-- Load data as raw chararray to safely filter headers
users_raw = LOAD 'hdfs://localhost:9000/movielens/input/users.csv' USING PigStorage(',')
AS (userId:chararray, gender:chararray, age:chararray, occupation:chararray, zip:chararray);
ratings_raw = LOAD 'hdfs://localhost:9000/movielens/input/ratings.csv' USING PigStorage(',')
AS (userId:chararray, movieId:chararray, rating:chararray, timestamp:chararray);

users_filtered = FILTER users_raw BY userId != 'userId' AND userId IS NOT NULL;
ratings_filtered = FILTER ratings_raw BY userId != 'userId' AND userId IS NOT NULL;

users = FOREACH users_filtered GENERATE
    (int)userId AS userId,
    gender AS gender,
    (int)age AS age,
    (int)occupation AS occupation,
    zip AS zip;

ratings = FOREACH ratings_filtered GENERATE
    (int)userId AS userId,
    (int)movieId AS movieId,
    (float)rating AS rating;

-- Join ratings with user demographic info
user_ratings = JOIN ratings BY userId, users BY userId;

-- Group by Gender and calculate average rating given
grp_gender = GROUP user_ratings BY users::gender;
gender_stats = FOREACH grp_gender GENERATE
    group AS gender,
    AVG(user_ratings.ratings::rating) AS avg_given_rating,
    COUNT(user_ratings) AS total_ratings;

-- Group by Age group and calculate stats
grp_age = GROUP user_ratings BY users::age;
age_stats = FOREACH grp_age GENERATE
    group AS age_group,
    AVG(user_ratings.ratings::rating) AS avg_given_rating,
    COUNT(user_ratings) AS total_ratings;

STORE gender_stats INTO 'hdfs://localhost:9000/movielens/pig_out2_gender' USING PigStorage(',');
STORE age_stats INTO 'hdfs://localhost:9000/movielens/pig_out2_age' USING PigStorage(',');
```

Execution:

```
C:\Users\jamal\Downloads\archive\script> pig -x mapreduce demographics_analysis.pig
2026-09-08 03:56:28,122 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-08 03:56:28,124 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-08 03:56:28,124 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-08 03:56:28,548 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-08 03:56:28,548 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig.1788818188530.log
2026-09-08 03:56:29,189 [main] INFO org.apache.pig.impl.util.Util - Default bootstrap file C:\Users\jamal\.pigbootstrap not found
2026-09-08 03:56:29,261 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-08 03:56:29,261 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-08 03:56:30,102 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-demographics_analysis.pig-9807d461-e7c2-498d-91d5-5cd25ead8730
2026-09-08 03:56:30,102 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-08 03:56:30,127 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:30,668 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:30,785 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:30,937 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:30,969 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:31,032 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.textoutputformat.separator is deprecated. Instead, use mapreduce.output.textoutputformat.separator
2026-09-08 03:56:31,074 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: HASH_JOIN, GROUP_BY, FILTER
2026-09-08 03:56:31,106 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 03:56:31,112 [main] INFO org.apache.pig.data.SchemaUpLeBackend - Key [pig.schematuple] was not set -- will not generate code.
2026-09-08 03:56:31,160 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter]}
2026-09-08 03:56:31,210 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for ratings_raw: $3
2026-09-08 03:56:31,286 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (G1 Old Gen) of size 1048576000 to monitor. collectionUsageThreshold = 734003200, usageThreshold = 734003200
2026-09-08 03:56:31,408 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-09-08 03:56:31,437 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-08 03:56:31,449 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
2026-09-08 03:56:31,477 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler$LastInputStreamingOptimizer - Rewrite: POPackage->POForEach to POPackage(JoinPackager)
2026-09-08 03:56:31,487 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size before optimization: 3
2026-09-08 03:56:31,489 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - Merged MR job 129 into MR job 172
2026-09-08 03:56:31,492 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - Merged MR job 122 into MR job 172
```

Output:

For age

```
1,3.549520414538238,27211
18,3.5075734460814227,183536
25,3.5452350615336385,395556
35,3.6181615352532375,199003
45,3.638061530735475,83633
50,3.714512346530556,72490
56,3.766632284682826,38780
```

For gender

```
F,3.6203660120110372,246440
M,3.5688785290984373,753769
```

6.4 Rating trends

Script:

```
-- Load raw data
ratings_raw = LOAD '/movielens/input/ratings.csv' USING PigStorage(',')
              AS (userId:chararray, movieId:chararray, rating:chararray, timestamp:chararray);

-- Step 1: Filter header & null values
ratings_filtered = FILTER ratings_raw BY userId != 'userId' AND userId IS NOT NULL;

-- Step 2: Convert Unix timestamp to year string and cast ratings
ratings_timed = FOREACH ratings_filtered GENERATE
  (float)rating AS rating,
  ToString(ToDate((long)timestamp * 1000L), 'yyyy') AS review_year;

-- Step 3: Group ratings by year
grouped_years = GROUP ratings_timed BY review_year;

-- Step 4: Calculate annual volume and average rating
yearly_stats = FOREACH grouped_years GENERATE
  group AS review_year,
  COUNT(ratings_timed) AS total_reviews,
  ROUND_TO(AVG(ratings_timed.rating), 2) AS avg_yearly_rating;

-- Step 5: Sort chronologically
sorted_years = ORDER yearly_stats BY review_year ASC;

-- Output Results
DUMP sorted_years;
STORE sorted_years INTO '/movielens/pig_out_trends' USING PigStorage(',');
```

Execution:

```
C:\Users\jamal\Downloads\archive\script> pig -x mapreduce rating_trends.pig
2026-09-08 13:18:35,795 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-08 13:18:35,795 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-08 13:18:35,795 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-08 13:18:36,368 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-08 13:18:36,368 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788851916334.log
2026-09-08 13:18:37,422 [main] INFO org.apache.pig.impl.util.Utils - Default bootup file C:\Users\jamal/.pigbootup not found
2026-09-08 13:18:37,505 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-08 13:18:37,505 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-08 13:18:38,742 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-rating_trends.pig-bda140fe-ab0c-4415-9cf2-49eec3aa73fb
2026-09-08 13:18:38,744 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-08 13:18:38,772 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 13:18:39,411 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 13:18:39,942 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: GROUP_BY,ORDER_BY,FILTER
2026-09-08 13:18:39,976 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 13:18:39,998 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-08 13:18:40,062 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeJoin, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter]}
2026-09-08 13:18:40,102 [main] INFO org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for ratings_raw: $1
2026-09-08 13:18:40,174 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (G1 Old Gen) of size 1048576000 to monitor. collectionUsageThreshold = 734003200, usageThreshold = 734003200
2026-09-08 13:18:40,280 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-09-08 13:18:40,318 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.CombinerOptimizerUtil - Choosing to move algebraic foreach to combiner
```

Output:

```
2000,904175,3.59
2001,68628,3.51
2002,24053,3.46
2003,3353,3.48
```

6.5 User_behavior

Script:

```
-- Load raw data directly from HDFS
ratings_raw = LOAD 'hdfs://localhost:9000/movielens/input/ratings.csv' USING PigStorage(',')
AS (userId:chararray, movieId:chararray, rating:chararray, timestamp:chararray);

-- Step 1: Filter header & nulls
ratings_filtered = FILTER ratings_raw BY userId != 'userId' AND userId IS NOT NULL;

-- Step 2: Cast user IDs and ratings
ratings = FOREACH ratings_filtered GENERATE
    (int)userId AS userId,
    (float)rating AS rating;

-- Step 3: Group by individual user
grouped_users = GROUP ratings BY userId;

-- Step 4: Calculate total reviews and average rating given per user
user_metrics = FOREACH grouped_users GENERATE
    group AS userId,
    COUNT(ratings) AS review_count,
    ROUND_TO(AVG(ratings.rating), 2) AS user_avg_rating;

-- Step 5: Segment users based on review activity count
user_segments = FOREACH user_metrics GENERATE
    userId,
    review_count,
    user_avg_rating,
    (
        review_count >= 100 ? 'Heavy Reviewer' :
        (review_count >= 30 ? 'Moderate Reviewer' : 'Casual Reviewer')
    ) AS engagement_tier;

-- Step 6: Group by segment tier to get overall summary stats
grouped_segments = GROUP user_segments BY engagement_tier;

segment_summary = FOREACH grouped_segments GENERATE
    group AS tier,
    COUNT(user_segments) AS total_users,
    ROUND_TO(AVG(user_segments.user_avg_rating), 2) AS tier_avg_rating;

-- STORE directly to HDFS path
STORE segment_summary INTO 'hdfs://localhost:9000/movielens/pig_out_user_segments' USING PigStorage(',');
```

Execution:

```
C:\Users\jamal\Downloads\archive\script> pig -x mapreduce user_behavior.pig
2026-09-08 04:09:38,336 INFO pig.ExecTypeProvider: Trying ExecType : LOCAL
2026-09-08 04:09:38,336 INFO pig.ExecTypeProvider: Trying ExecType : MAPREDUCE
2026-09-08 04:09:38,336 INFO pig.ExecTypeProvider: Picked MAPREDUCE as the ExecType
2026-09-08 04:09:38,728 [main] INFO org.apache.pig.Main - Apache Pig version 0.17.0 (r1797386) compiled Jun 02 2017, 15:41:58
2026-09-08 04:09:38,728 [main] INFO org.apache.pig.Main - Logging error messages to: C:\hadoop\logs\pig_1788818978717.log
2026-09-08 04:09:39,364 [main] INFO org.apache.pig.impl.util.Utils - Default bootstrap file C:\Users\jamal/.pigbootstrap not found
2026-09-08 04:09:39,453 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-09-08 04:09:39,453 [main] INFO org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://localhost:9000
2026-09-08 04:09:40,186 [main] INFO org.apache.pig.PigServer - Pig Script ID for the session: PIG-user_behavior.pig-d8218d8c-3ae8-49c5-85bf-b1e6a6966315
2026-09-08 04:09:40,186 [main] WARN org.apache.pig.PigServer - ATS is disabled since yarn.timeline-service.enabled set to false
2026-09-08 04:09:40,220 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:09:40,693 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:09:40,883 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:09:40,955 [main] WARN org.apache.pig.newplan.BaseOperatorPlan - Encountered Warning IMPLICIT_CAST_TO_LONG 2 time(s).
2026-09-08 04:09:40,955 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapred.textoutputformat.separator is deprecated. Instead, use mapreduce.output.textoutputformat.separator
2026-09-08 04:09:41,012 [main] INFO org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: GROUP_BY,FILTER
2026-09-08 04:09:41,042 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - mapreduce.job.counters.limit is deprecated. Instead, use mapreduce.job.counters.max
2026-09-08 04:09:41,042 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schemaTuple] was not set.... will not generate code.
2026-09-08 04:09:41,090 [main] INFO org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NestedLimitOptimizer, PartitionFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter]}
2026-09-08 04:09:41,122 [main] INFO org.apache.pig.impl.util.SpillableMemoryManager - Selected heap (G1 Old Gen) of size 1048576000 to monitor. collectionUsageThreshold = 734003200, usageThreshold = 734003200
```

Output:

```
Heavy Reviewer,2945,3.63
Casual Reviewer,751,3.76
Moderate Reviewer,2344,3.78
```

7. Conclusion

This project presents a complete Hadoop-oriented workflow for MovieLens data analysis. The Java MapReduce programs demonstrate explicit mapper, reducer, combiner, cache, and map-only patterns. The Pig scripts demonstrate concise data-flow analysis over ratings, movies, and users, including joins, aggregation, ranking, temporal analysis, demographic comparison, and user segmentation. The measured dataset profile confirms that the project operates on a substantial input: more than one million rating events. Its design is therefore appropriate for demonstrating why distributed processing is useful, while its modular output directories make individual analyses easy to validate and reuse.